

Efficient Graph Deep Learning in TensorFlow with `tf_geometric`

Jun Hu¹, Shengsheng Qian^{1,2}, Quan Fang^{1,2}, Youze Wang³

Quan Zhao³, Huaiwen Zhang^{1,2,4}, Changsheng Xu^{1,2,*}

¹National Laboratory of Pattern Recognition, Institute of Automation, Chinese Academy of Sciences

²School of Artificial Intelligence, University of Chinese Academy of Sciences

³Hefei University of Technology

⁴Key Laboratory of Artificial Intelligence Scenario Application and Intelligent System Evaluation (China Center for Information Industry Development), Ministry of Industry and Information Technology

hujunxianligong@gmail.com, {shengsheng.qian, qfang}@nlpr.ia.ac.cn

{youze.wang, 2019111002}@mail.hfut.edu.cn, {huaiwen.zhang, csxu}@nlpr.ia.ac.cn

ABSTRACT

We introduce `tf_geometric`¹, an efficient and friendly library for graph deep learning, which is compatible with both TensorFlow 1.x and 2.x. It provides kernel libraries for building Graph Neural Networks (GNNs) as well as implementations of popular GNNs. The kernel libraries consist of infrastructures for building efficient GNNs, including graph data structures, graph map-reduce framework, graph mini-batch strategy, etc. These infrastructures enable `tf_geometric` to support single-graph computation, multi-graph computation, graph mini-batch, distributed training, etc.; therefore, `tf_geometric` can be used for a variety of graph deep learning tasks, such as node classification, link prediction, and graph classification. Based on the kernel libraries, `tf_geometric` implements a variety of popular GNN models. To facilitate the implementation of GNNs, `tf_geometric` also provides some other libraries for dataset management, graph sampling, etc. Different from existing popular GNN libraries, `tf_geometric` provides not only Object-Oriented Programming (OOP) APIs, but also Functional APIs, which enable `tf_geometric` to handle advanced tasks such as graph meta-learning. The APIs are friendly and suitable for both beginners and experts.

CCS CONCEPTS

• **Information systems** → *Computing platforms*.

KEYWORDS

Graph Neural Networks, Graph Deep Learning

ACM Reference Format:

Jun Hu, Shengsheng Qian, Quan Fang, Youze Wang, Quan Zhao, Huaiwen Zhang, and Changsheng Xu. 2021. Efficient Graph Deep Learning in TensorFlow with `tf_geometric`. In *Proceedings of the 29th ACM International Conference on Multimedia (MM '21)*, October 20–24, 2021, Virtual Event, China. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3474085.3478322>

*Corresponding authors

¹https://github.com/CrawlScript/tf_geometric

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '21, October 20–24, 2021, Virtual Event, China

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8651-7/21/10...\$15.00

<https://doi.org/10.1145/3474085.3478322>

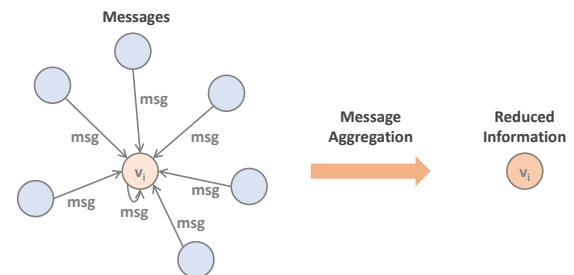


Figure 1: An Example of Message Aggregation

1 INTRODUCTION

In recent years, Graph Neural Networks (GNNs) emerge as powerful tools for deep learning on graphs, which aims to understand the semantics of graph data. GNNs have been successfully applied to a variety of tasks in different fields, such as multi-modal question answering [3, 7], social multi-media understanding [12], and multi-model reasoning [14].

Due to the properties of graph data, such as sparsity and irregularity, it is challenging to implement efficient and friendly GNN libraries. It is known that the most challenging problem for implementing GNNs is the aggregation of graph data. Aggregation is the fundamental operation for most GNNs. There are mainly two types of aggregation in GNNs: message aggregation (Fig. 1) and graph pooling (Fig. 2). The two main types of aggregation allow researchers and engineers to design complex GNNs, and thus proper solutions for aggregation on graphs are imperative for building elegant GNN models. However, it is non-trivial to design proper aggregation solutions for sparse and irregular graph data. Most intuitive solutions, such as padding and masking, usually suffer from the memory and efficiency problem, whereas many efficient solutions, such as sparse matrix multiplication, require a lot of complex tricks to accomplish advanced aggregations.

We develop `tf_geometric`, an efficient and friendly GNN library for deep learning on sparse and irregular graph data, which is compatible with both TensorFlow 1.x and 2.x. `tf_geometric` provides a unified solution for GNNs, which mainly consists of kernel libraries for building graph neural networks and implementations of popular GNNs. The kernel libraries contain infrastructures for building efficient GNNs, including graph data structures, graph map-reduce framework, etc. In particular, the graph data structures and map-reduce framework provide an elegant and efficient way for aggregation on graphs. The kernel libraries enable `tf_geometric`

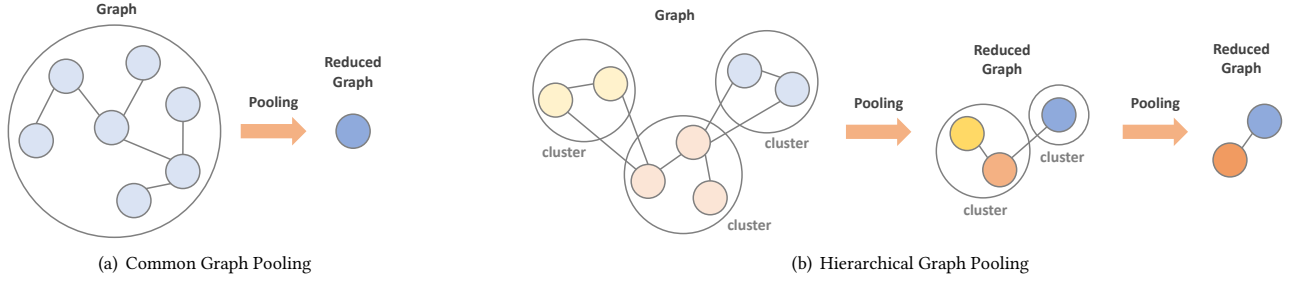


Figure 2: Examples of Graph Pooling

to support single-graph computation, multi-graph computation, graph mini-batch, distributed training, etc. Based on the kernel libraries, a variety of popular GNN models are implemented as APIs. To facilitate the implementation of GNNs, `tf_geometric` also provides some other libraries for dataset management, graph sampling, etc. Different from existing popular GNN libraries, `tf_geometric` provides not only OOP APIs, but also Functional APIs, which enable `tf_geometric` to handle advanced graph deep learning tasks such as graph meta-learning. The APIs of `tf_geometric` are friendly, and they are suitable for both beginners and experts. `tf_geometric` is available on GitHub². The features of `tf_geometric` are thoroughly documented³ and a collection of accompanying tutorials and examples are also provided in the documentation.

2 OVERVIEW

`tf_geometric` mainly consists of kernel libraries for building graph neural networks and implementations of popular GNNs. Besides, some other libraries such as dataset management and graph sampling are also provided to facilitate the implementation of GNNs.

2.1 Kernel Libraries

The framework of the kernel libraries is shown in Fig. 3. As shown in Fig. 3, the kernel libraries consist of several fundamental components as infrastructures for building efficient GNNs, which enable `tf_geometric` to support single-graph computation, multi-graph computation, graph mini-batch, distributed training, etc.

2.1.1 Graph Data Structure. `tf_geometric` has two core graph data structures (classes): **Graph** and **BatchGraph**, which are used to model a single graph and a batch of graphs, respectively.

A **Graph** object $graph = (x, edge_index, edge_weight, y)$ is used to model a single graph, where x , $(edge_index, edge_weight)$, and y correspond to node features, adjacency matrix, and label information, respectively. The adjacency matrix is presented as a sparse matrix in coordinate (COO) format, which can benefit many aggregation operations in GNNs. For each **Graph** object, the GNN models in `tf_geometric` can take advantage of parallelism capabilities of deep learning frameworks to efficiently process graph data. However, due to the limitation of most deep learning frameworks, it is difficult to process information in different graph objects simultaneously. Therefore, for tasks dealing with multiple graphs, `tf_geometric` introduces the **BatchGraph** class, which allow the GNN models to process information in multiple graphs in parallel.

A **BatchGraph** object stores the information of multiple graph objects, and it enables the parallel processing of data from different graphs by virtualizing multiple graphs as a single graph (Fig. 4).

Note that the `BatchGraph` class is a subclass of the `Graph` class, and therefore applying GNN operations on a `BatchGraph` automatically enables the parallel processing of multiple graphs, which brings dramatic performance improvement in computational efficiency.

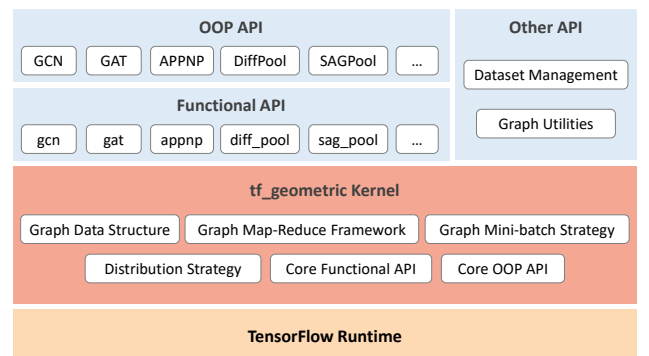
2.1.2 Graph Map-Reduce Framework. Many complex GNNs can be considered as a combination of simple map-reduce operations on graphs. Usually, map and reduce operations correspond to transformation and aggregation operations on graphs. The `tf_geometric` kernel provides a graph map-reduce framework, including basic and advanced map and reduce operations on graphs and graph map-reduce workflows. Fig. 5 shows an example of map-reduce workflow, which computes the normalized attention scores for a Graph Attention Network (GAT).

2.1.3 Graph Mini-batch Strategy. As with general deep learning models, GNNs can benefit from mini-batch training and inference of graphs. Given a batch of graphs, `tf_geometric` combine them into a `BatchGraph` and apply GNNs on it.

2.1.4 Distribution Strategy. To take advantage of the powerful distribution ability of TensorFlow, all the GNN models in `tf_geometric` are implemented as standard TensorFlow models, which can be distributed with minimal code changes on the data processing. The distribution of `tf_geometric` GNN models can be easily handled by TensorFlow distribution strategies, and the model can be distributed in different ways by choosing different distribution strategies.

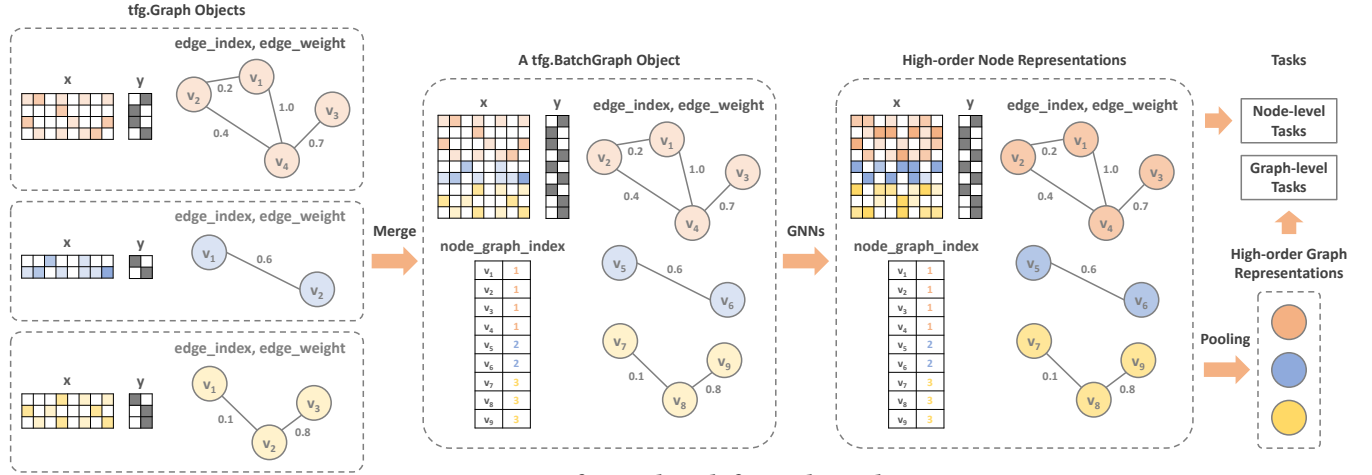
2.1.5 Core OOP APIs and Functional APIs. `tf_geometric` provides both **Object Oriented Programming (OOP)** and **Functional APIs**, with which users can built advanced graph deep learning models.

OOP APIs are class-level interfaces for graph neural networks. The GNN classes in `tf_geometric` are implemented as standard TensorFlow models by subclassing the `tf.keras.Model` class, where each GNN class defines how to maintain the model parameters and the computational process.

Figure 3: The Framework of `tf_geometric`

²https://github.com/CrawlScript/tf_geometric

³<https://tf-geometric.readthedocs.io>

Figure 4: `tf.Graph` and `tf.BatchGraph`

Functional APIs provide function-level interfaces for graph neural networks. Functional APIs are functions that implement GNN operations. Instead of using fixed tensors as model parameters in OOP APIs, functional APIs can dynamically use different tensors as model parameters for each call. This feature of functional APIs is critical for advanced tasks that require complex maintenance strategies of model parameters, such as graph meta-learning.

2.2 Implementation of Popular GNN Models

Based on the kernel libraries, `tf_geometric` implements a variety of popular GNN models for different tasks. To avoid redundancy, all the GNN models in `tf_geometric` are first implemented as Functional APIs, and the OOP APIs are just wrappers of the corresponding Functional APIs. Besides, `tf_geometric` also provides demos that reproduce the performance of GNNs reported in the literature.

2.3 Dataset Management Mechanism

`tf_geometric` provides a lot of ready-to-use public benchmark datasets (built-in datasets) that are frequently used in graph deep learning research. Users can also customize their datasets by simply

subclassing built-in abstract dataset classes. The built-in abstract dataset classes manage the workflow of dataset processing and already encapsulate the implementation of general processes, such as downloading, file management, and caching.

3 COMPARISON TO OTHER GNN LIBRARIES

In recent years, several GNN libraries have been developed for different deep learning frameworks. Among them, popular libraries such as PyTorch Geometric (PyG)⁴ [1] and Deep Graph Library (DGL)⁵ [11] have been widely used by researchers to deal with graph deep learning in different fields. They provide extensible OOP APIs and implement a variety of GNN classes, with which users can easily handle general graph deep learning tasks. As mentioned before, OOP APIs are insufficient for several advanced tasks such as graph meta-learning. Different from these GNN libraries, `tf_geometric` provide not only OOP APIs, but also Functional APIs, which can be used to deal with advanced graph deep learning tasks. Moreover, popular GNN libraries for TensorFlow such as Spektral⁶ [2] and StellarGraph⁷ usually only support TensorFlow 2.x, whereas `tf_geometric` is compatible with both TensorFlow 1.x and 2.x. Furthermore, `tf_geometric` provides a flexible and friendly caching system that can speed up some GNNs, while existing GNN libraries do not support caching or only support caching for few special cases. For example, PyG only supports layer-level caching for GCN, which means that each PyG GCN layer with caching enabled is bound to a constant graph structure and usually it can only be used for transductive learning tasks on a single graph. Instead, the `tf_geometric` GCN adopts a graph-level caching mechanism, and it can cache for different graph structures with different GCN normalization configurations.

4 EMPIRICAL EVALUATION

4.1 Tasks and Evaluation Metrics

We evaluate several `tf_geometric` models on two different tasks.

Node Classification We first conduct experiments on a semi-supervised node classification task with three benchmark datasets:

⁴https://github.com/rusty1s/pytorch_geometric

⁵<https://github.com/dmlc/dgl>

⁶<https://github.com/danielegrattarola/spektral>

⁷<https://github.com/stellargraph/stellargraph>

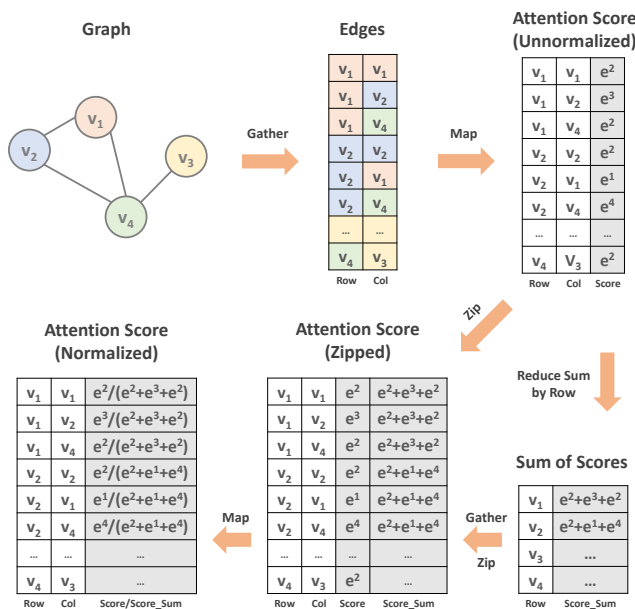


Figure 5: Graph Map-Reduce Framework

Table 1: Performance on Node Classification Tasks.

Model \ Dataset	Cora	CiteSeer	Pubmed
GCN	81.7 $\pm$ 0.5	71.0 $\pm$ 0.8	79.1 $\pm$ 0.6
GAT	83.0 $\pm$ 0.8	72.9 $\pm$ 0.8	79.0 $\pm$ 0.2
SGC	81.1 $\pm$ 0.0	72.1 $\pm$ 0.0	79.2 $\pm$ 0.1
APPNP	83.6 $\pm$ 0.7	72.2 $\pm$ 0.9	79.0 $\pm$ 0.7
DGI	82.4 $\pm$ 0.5	72.2 $\pm$ 0.5	78.0 $\pm$ 0.6

Table 2: Performance on Graph Classification Tasks.

Model \ Dataset	NCI1	NCI109	PROTEINS
Mean-Max Pool	76.03 $\pm$ 0.7	75.62 $\pm$ 0.7	74.18 $\pm$ 0.6
Set2Set	72.01 $\pm$ 0.6	69.54 $\pm$ 0.6	71.69 $\pm$ 0.6
SortPool	74.65 $\pm$ 0.6	73.22 $\pm$ 0.7	68.09 $\pm$ 2.6
DiffPool	75.27 $\pm$ 0.8	73.73 $\pm$ 0.6	74.23 $\pm$ 1.0
SAGPool _h	69.41 $\pm$ 1.1	69.12 $\pm$ 0.5	73.33 $\pm$ 0.8

Cora, CiteSeer, and Pubmed. We evaluate GCN [4], GAT [8], SGC [13], APPNP [5], and DGI [9] on the task. For the benchmark datasets, we use the same dataset splits as in [4], where each dataset is split into a train set, a test set, and a validation set. The validation set is only used for early stopping.

Graph Classification We also evaluate `tf_geometric` on a graph classification task with three benchmark datasets: NCI1, NCI109, and PROTEINS. We evaluate several graph pooling models, including Mean-Max Pool, Set2Set [10], SortPool [16], DiffPool [15], and SAGPool_h [6]. The Mean-Max Pool is a naive graph pooling model, which obtains graph representations by concatenating the mean pooling and max pooling results of GCNs. These classification accuracy scores of these models are evaluated on three benchmark datasets using 10-fold cross-validation, where a training fold is randomly sampled as the validation set. As with the node classification task, the validation set is only used for early stopping.

4.2 Performance

The model performance (accuracy scores) on node classification is reported in Table 1. The results show that the GNN models provided by `tf_geometric` can achieve competitive performance with the official implementations. For the graph classification task, the results (accuracy scores) are listed in Table 2. Since the architectures of some models are adjusted, the model performance is sometimes better than that reported in the literature. Note that by optimizing the architecture, the naive Mean-Max Pool outperforms some other graph pooling models in some cases.

5 CONCLUSIONS

We introduce `tf_geometric`, an efficient and friendly library for graph deep learning, which is compatible with both TensorFlow 1.x and 2.x. `tf_geometric` provides kernel libraries for building graph neural networks as well as implementations of popular GNNs. In particular, the kernel libraries consist of infrastructures for building efficient GNNs, which enable `tf_geometric` to support single-graph computation, multi-graph computation, distributed training, etc. `tf_geometric` exposes both OOP APIs and Functional APIs, with which users can deal with advanced graph deep learning tasks. We are actively working to further optimize the kernel libraries and integrate more existing GNN models for `tf_geometric`.

ACKNOWLEDGMENTS

This work was supported in part by the National Key Research and Development Program of China (No. 2017YFB1002804), the National Natural Science Foundation of China under Grants, 62036012, 62072456, 61720106006, 61572503, 61802405, 61872424, 61702509, 61832002, 61936005, and U1705262, the Key Research Program of Frontier Sciences, CAS (No. QYZDJ-SSW-JSC039), the CCF-Tencent Open Fund, the Open Research Projects of Zhejiang Lab (No. 2021KE0AB05).

REFERENCES

- [1] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. *CoRR* abs/1903.02428 (2019).
- [2] Daniele Grattarola and Cesare Alippi. 2021. Graph Neural Networks in TensorFlow and Keras with Spektral [Application Notes]. *IEEE Comput. Intell. Mag.* 16, 1 (2021), 99–106.
- [3] Deng Huang, Peihao Chen, Runhao Zeng, Qing Du, Minghui Tan, and Chuang Gan. 2020. Location-Aware Graph Convolutional Networks for Video Question Answering. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, New York, NY, USA, February 7–12, 2020*. AAAI Press, 11021–11028.
- [4] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net.
- [5] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019*. OpenReview.net.
- [6] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. 2019. Self-Attention Graph Pooling. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9–15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 3734–3743.
- [7] Raed Saqur and Karthik Narasimhan. 2020. Multimodal Graph Networks for Compositional Generalization in Visual Question Answering. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual*.
- [8] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 – May 3, 2018, Conference Track Proceedings*. OpenReview.net.
- [9] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2019. Deep Graph Infomax. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019*. OpenReview.net.
- [10] Oriol Vinyals, Samy Bengio, and Manjunath Kudlur. 2016. Order Matters: Sequence to sequence for sets. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings*.
- [11] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, Ziyue Huang, Qipeng Guo, Hao Zhang, Haibin Lin, Junbo Zhao, Jinyang Li, Alexander J. Smola, and Zheng Zhang. 2019. Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs. *CoRR* abs/1909.01315 (2019).
- [12] Youze Wang, Shengsheng Qian, Jun Hu, Quan Fang, and Changsheng Xu. 2020. Fake News Detection via Knowledge-driven Multimodal Graph Convolutional Networks. In *Proceedings of the 2020 International Conference on Multimedia Retrieval, ICMR 2020, Dublin, Ireland, June 8–11, 2020*. ACM, 540–547.
- [13] Felix Wu, Amauri H. Souza Jr., Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9–15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 6861–6871.
- [14] Peixi Xiong, Huayi Zhan, Xin Wang, Baivav Sinha, and Ying Wu. 2019. Visual Query Answering by Entity-Attribute Graph Matching and Reasoning. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16–20, 2019*. Computer Vision Foundation / IEEE, 8357–8366.
- [15] Zhitaoying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. 2018. Hierarchical Graph Representation Learning with Differentiable Pooling. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3–8, 2018, Montréal, Canada*. 4805–4815.
- [16] Muhun Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An End-to-End Deep Learning Architecture for Graph Classification. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, AAAI-18, New Orleans, Louisiana, USA, February 2–7, 2018*. AAAI Press, 4438–4445.